



Цифровые технологии управления
транспортными процессами

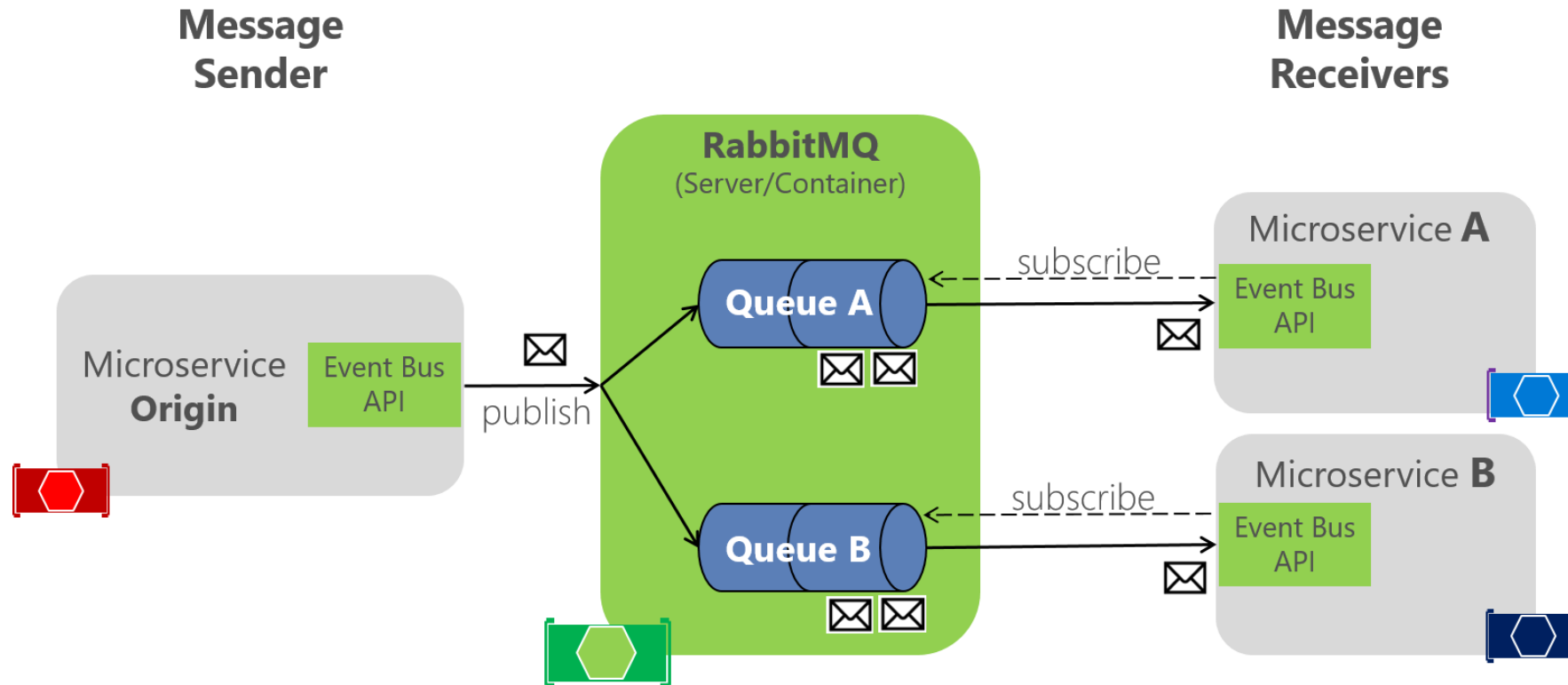
Сервис-ориентированное
программирование

RabbitMQ

RabbitMQ+Java



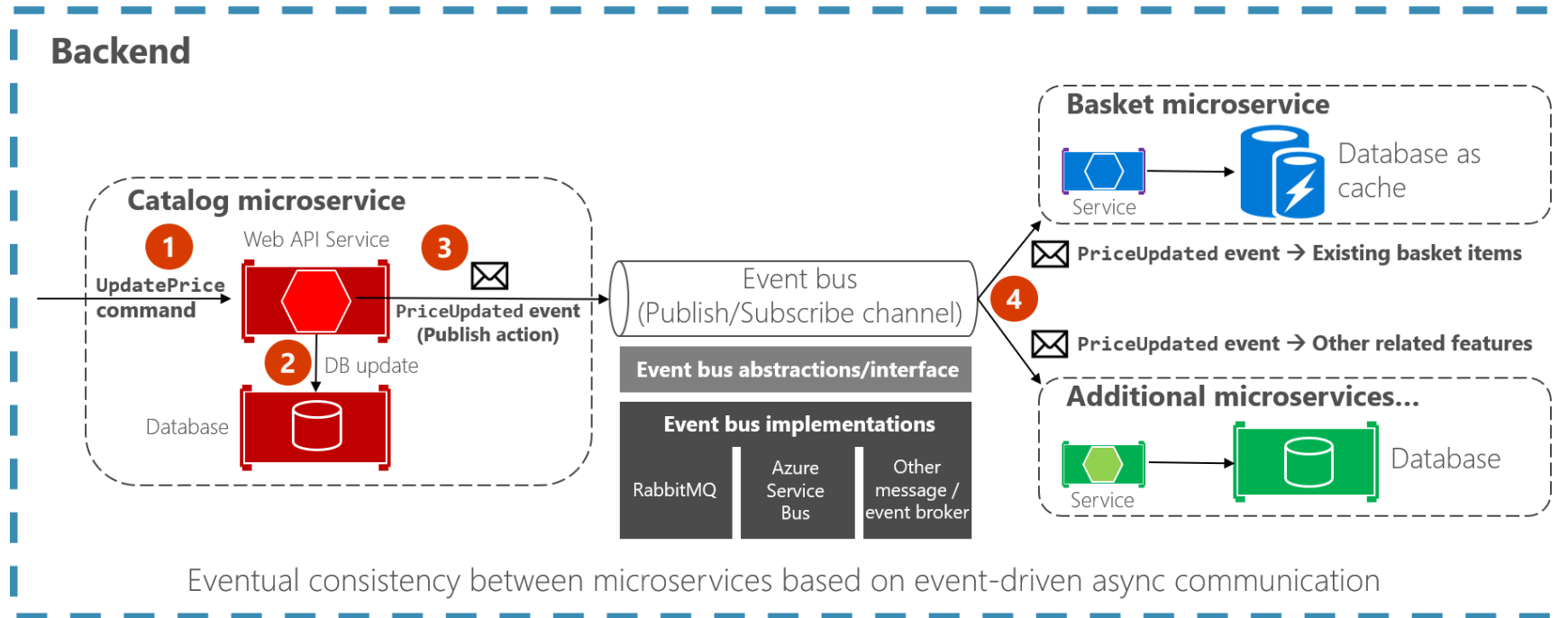
Для чего он нужен?





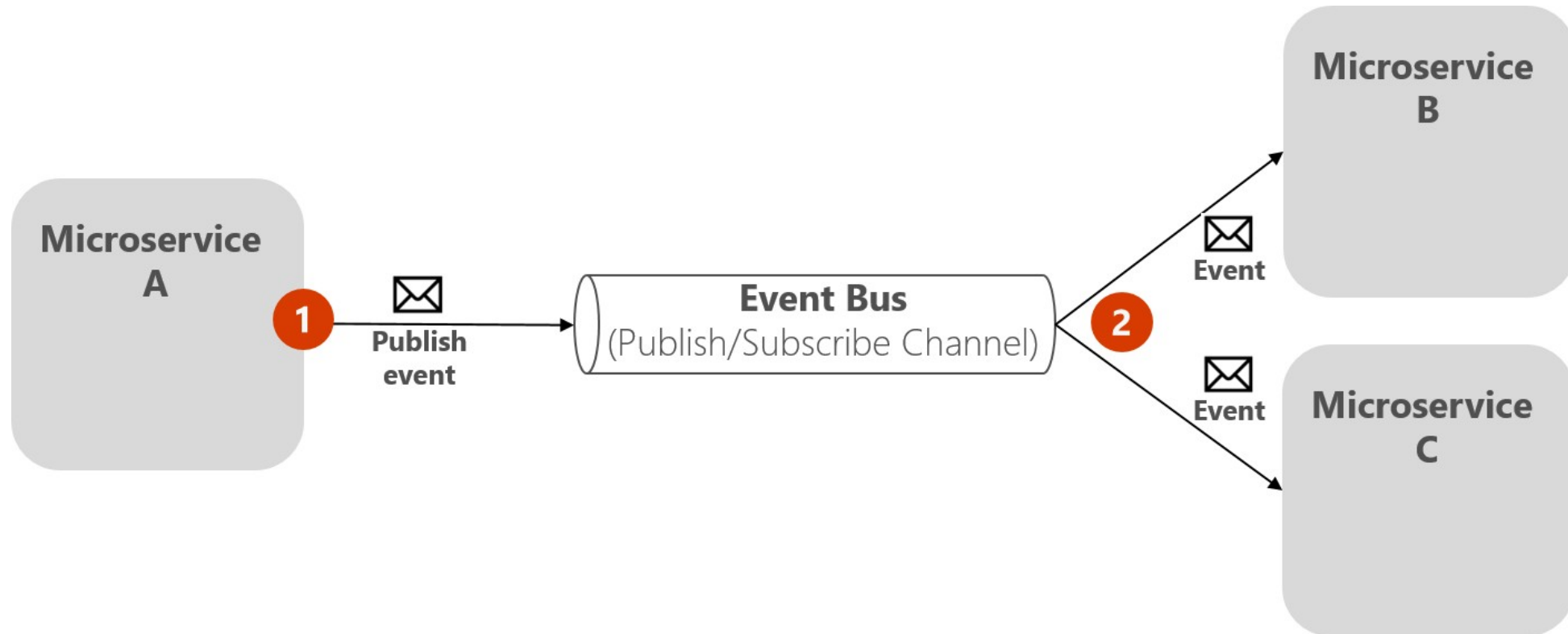
Для чего он нужен?

Implementing asynchronous event-driven communication with an event bus





Для чего он нужен?





Паттерны

- Publish/Subscribe
 - [https://docs.microsoft.com/ru-RU/previous-versions/msp-n-p/ff649664\(v=pandp.10\)](https://docs.microsoft.com/ru-RU/previous-versions/msp-n-p/ff649664(v=pandp.10))
- Observer (Наблюдатель)
 - https://en.wikipedia.org/wiki/Observer_pattern

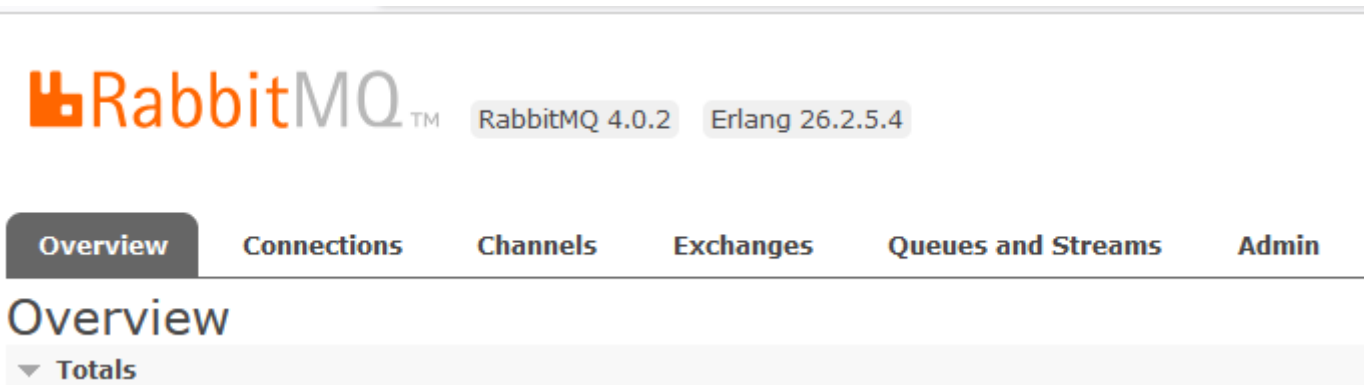


Docker

- Самый простой способ запустить RabbitMQ, загрузить его в докер контейнере

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:management
```

- Проверить можно перейдя по ссылке <http://localhost:15672>
 - Логин/Пароль
 - guest/guest





vhost

 **RabbitMQ**™

RabbitMQ 4.0.2 Erlang 26.2.5.4

Refreshed 2024-10-12 15:40:07

Refresh every 5 seconds

Virtual host

All

Cluster **rabbit@b12c84950afc**

User **guest**

Log out

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

▼ All virtual hosts

Filter: ☐ Regex ?

1 item, page size up to

Overview			Messages			Network		Message rates	
Name	Users ?	State	Ready	Unacked	Total	From client	To client	publish	deliver / get
/	guest	running	?	?	?				

▼ Add a new virtual host

Name:

Description:

Tags:

Default Queue Type:

Classic

Add virtual host

Virtual Hosts

Feature Flags

Deprecated Features

Policies

Limits

Cluster

HTTP API

Documentation

Tutorials

New releases

Commercial edition

Commercial support

Discussions

Discord

Plugins

GitHub



Java

- Создадим несколько проектов на Java:
 - Spring Boot проект для producer'a
 - Консольное приложение как consumer



Spring Boot

Server URL: start.spring.io ⚙️

Name:

Location:

Project will be created in: ~\Downloads\SpringRabbitMQDemo

☐ Create Git repository

Language:

Type:

Group:

Artifact:

Package name:

JDK:

Java:

Packaging:

Added dependencies:

- × Spring Web
- × Spring for RabbitMQ



Spring Boot

application.properties x

```
1 spring.rabbitmq.host=localhost
2 spring.rabbitmq.virtual-host=vhost
3 spring.rabbitmq.password=guest
4 spring.rabbitmq.username=guest
5
6 server.port=8081
```



RabbitMQConfiguration

```
@Configuration
public class RabbitMQConfiguration {
    static final String queueName = "firstQueue"; 1 usage
    static final String exchangeName = "testExchange"; 1 usage

    @Bean
    public Queue myQueue() { return new Queue(queueName, durable: false); }

    @Bean
    Exchange exchange() { return new TopicExchange(exchangeName, durable: false, autoDelete: false); }

    @Bean
    Binding binding(Queue queue, Exchange exchange) {
        return BindingBuilder.bind(queue).to(exchange).with( routingKey: "my.key").noargs();
    }
}
```



SendController

```
@RestController
@RequestMapping("/send")
public class SendController {
    private final RabbitTemplate rabbitTemplate; 2 usages
    private static Long counter = 0L; no usages
    static final String exchangeName = "testExchange"; 1 usage

    public SendController(RabbitTemplate rabbitTemplate) {
        this.rabbitTemplate = rabbitTemplate;
    }

    @PostMapping("/message")
    public ResponseEntity<String> sendMessageToQ1(@RequestBody String message) {
        rabbitTemplate.convertAndSend(exchangeName, routingKey: "my.key", message);

        return ResponseEntity.ok().body("Send to Q1!");
    }
}
```



Создаем consumer

Server URL: start.spring.io

Name:

Location:

Project will be created in: ~\Downloads\rabbitmq-master\RabbitMQConsum

☐ Create Git repository

Language:

Type:

Group:

Artifact:

Package name:

JDK:

Java:

Packaging:

Added dependencies:

× Spring for RabbitMQ



Создаем consumer

⚙ application.properties ×

```
1 spring.application.name=RabbitMQConsumer
2 spring.rabbitmq.host=localhost
3 spring.rabbitmq.virtual-host=vhost
4 spring.rabbitmq.password=guest
5 spring.rabbitmq.username=guest
6
```



Создаем consumer

```
@SpringBootApplication
public class RabbitMqConsumerApplication {
    static final String queueName = "firstQueue"; 2 usages

    @Bean
    public Queue myQueue() { return new Queue(queueName, durable: false); }

    @RabbitListener(queues = queueName)
    public void listen(String message) { System.out.println("Message read from firstQueue : " + message); }

    public static void main(String[] args) { SpringApplication.run(RabbitMqConsumerApplication.class, args); }
}
```



api.http

```
@base_address = http://localhost:8081
```

```
###
```

```
POST {{base_address}}/send/message
```

```
Content-Type: application/json
```

```
{  
  "message": "Hello, World!"  
}
```



Отправлено

The screenshot shows an API client interface. On the left, under 'API HTTP Request', a 'POST' request to 'api' is highlighted, showing a status of 200 (133 ms). Below it, a message indicates 'POST /message HTTP/1.1 Status: Error in the request'. On the right, the details of the request are shown: 'POST http://localhost:8081/send/message', 'Show Request' button, 'HTTP/1.1 200', and headers including 'Content-Type: text/plain; charset=UTF-8...'. At the bottom, it says 'Send to Q1!' and 'Response code: 200; Time: 133ms (133 ms); Content length: 11 bytes (11 B)'.

The screenshot shows a 'Run' window for 'RabbitMqConsumerApplication'. The 'Console' tab is active, displaying several log entries from 'RabbitMQConsumer' at 2024-10-12T15:40:55. The logs show a sequence of messages being read from a queue. The final log entry shows a message with the content 'Hello, World!'.

```
2024-10-12T15:40:55.326+03:00 INFO 8640 --- [RabbitMQConsumer] [
2024-10-12T15:40:55.327+03:00 INFO 8640 --- [RabbitMQConsumer] [
2024-10-12T15:40:55.859+03:00 INFO 8640 --- [RabbitMQConsumer] [
2024-10-12T15:40:55.889+03:00 INFO 8640 --- [RabbitMQConsumer] [
2024-10-12T15:40:55.893+03:00 INFO 8640 --- [RabbitMQConsumer] [
2024-10-12T15:40:55.921+03:00 INFO 8640 --- [RabbitMQConsumer] [
Message read from firstQueue : {
  "message": "Hello, World!"
}
```



Отправлено



RabbitMQ 4.0.2 Erlang 26.2.5.4

Refreshed 2024-10-12 15:42:35 Refresh every 5 seconds

Virtual host All

Cluster rabbit@b12c84950afc

User guest Log out

Overview Connections Channels Exchanges Queues and Streams

Admin

▼ All queues (1)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Displaying 1 item , page size up to: 100

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
vhost	firstQueue	classic	Args	running	0	0	0	0.00/s	0.00/s	0.00/s	

► Add a new queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins

GitHub



Отправлено

▼ Consumers (1)

Channel ?	Consumer tag	Ack required	Exclusive	Prefetch count	Active ?	Activity status	Consumer Timeout	Arguments
172.17.0.1:55850 (1)	amq.ctag-JMDknea0iWnhi0HaF2lekQ	•	○	250	•	up	1800000	

▼ Bindings (2)

From	Routing key	Arguments	
(Default exchange binding)			
testExchange	my.key		Unbind



This queue



Publish message

▼ Publish message

Message will be published to the default exchange with routing key **firstQueue**, routing it to this queue.

Delivery mode: 1 - Non-persistent ▼

Headers: ? = String ▼

Properties: ? =

Payload:

Payload encoding: String (default) ▼



Publish message

▼ Publish message

Message will be published to the default exchange with routing key **firstQueue**, routing it to this queue.

Delivery mode: 1 - Non-persistent ▼

Headers: ? = String ▼

Properties: ? =

Payload: Сообщение из админки!

Payload encoding: String (default) ▼

Publish message

```
2024-10-12T15:40:55.921+03:00 INFO 8640 --- [RabbitMQConsu
Message read from firstQueue : {
  "message": "Hello, World!"
}
Message read from firstQueue : Сообщение из админки!
```



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование

RabbitM
Q

RabbitMQ+.NET



Проект и pub/sub

- RabbitMQ
 - <https://www.rabbitmq.com/>
- Развернем в качестве отдельного сервиса
 - Локально для теста стоит развернуть в docker
 - [Install Docker Engine | Docker Documentation](#)
- Воспользуемся библиотекой:
 - [GitHub - EasyNetQ/EasyNetQ: An easy to use .NET API for RabbitMQ](#)
 - [EasyNetQ - An easy .NET API for RabbitMQ](#)
- В продакшене можно использовать <https://particular.net/nservicebus> и <https://masstransit-project.com/> для реализации агностик промежуточного слоя для обмена сообщениями как с помощью RabbitMQ, так и облачными альтернативами - Azure Service Bus, Amazon SQS/SNS, смотря где разворачиваете и сами ли.



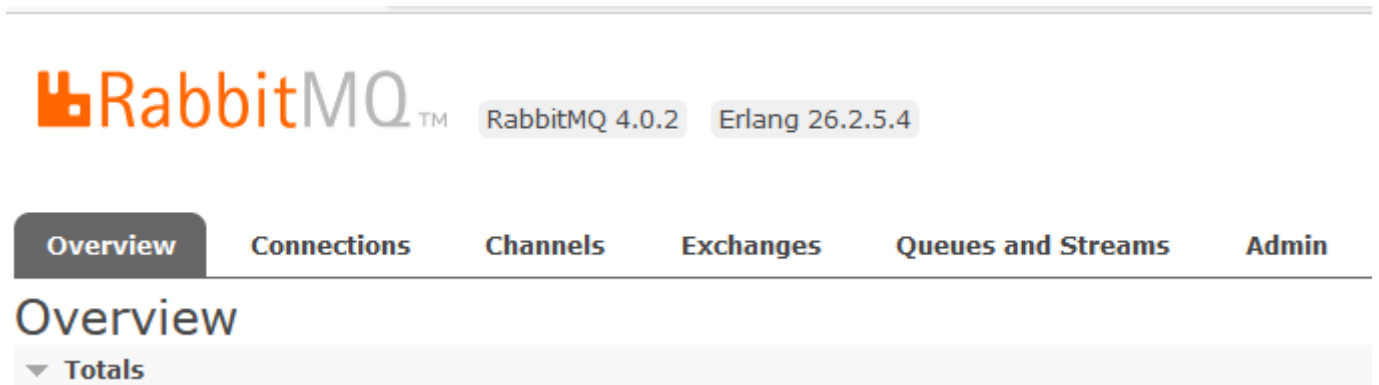


Docker

- Самый простой способ запустить RabbitMQ, запустить его в докер контейнере

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:management
```

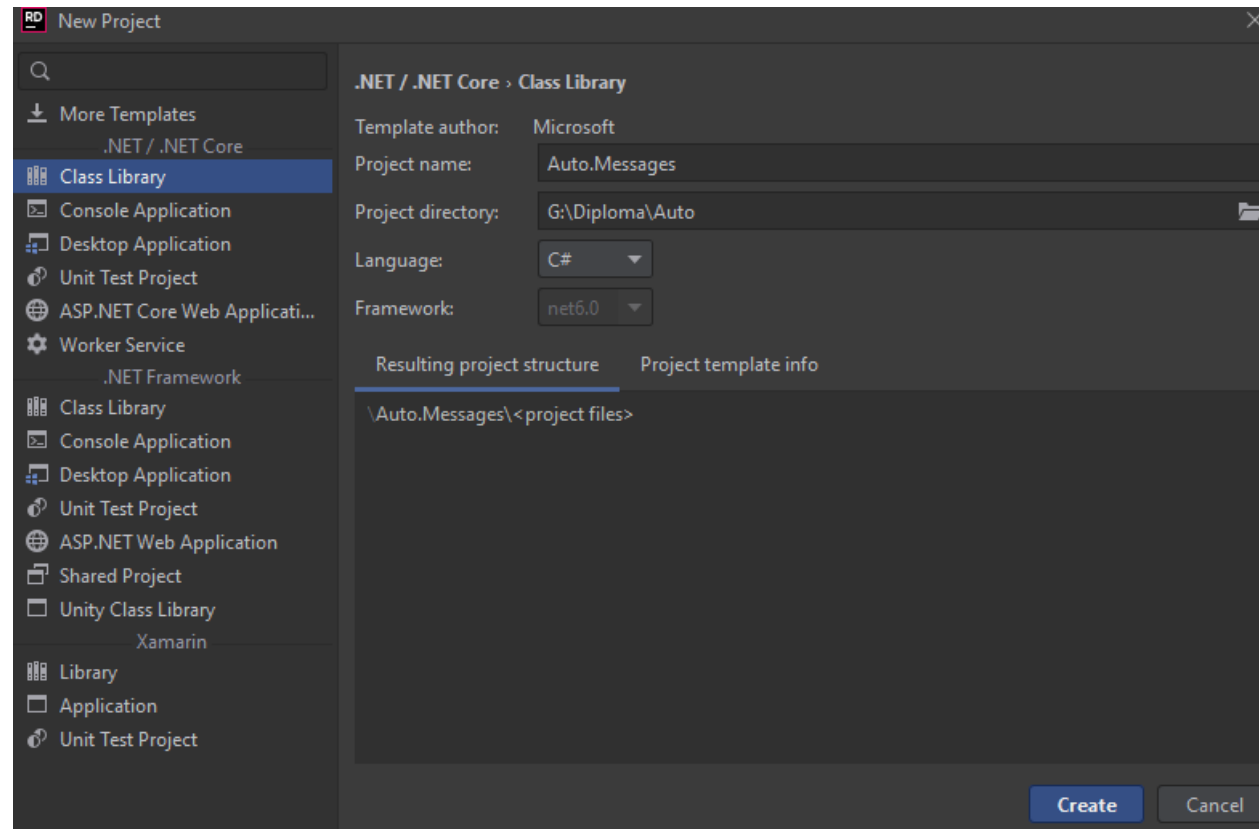
- Проверить можно перейдя по ссылке <http://localhost:15672>
 - Логин/Пароль
 - guest/guest





Создадим библиотеку классов с сообщениями

Auto.Messages:





Добавим классы с сообщениями

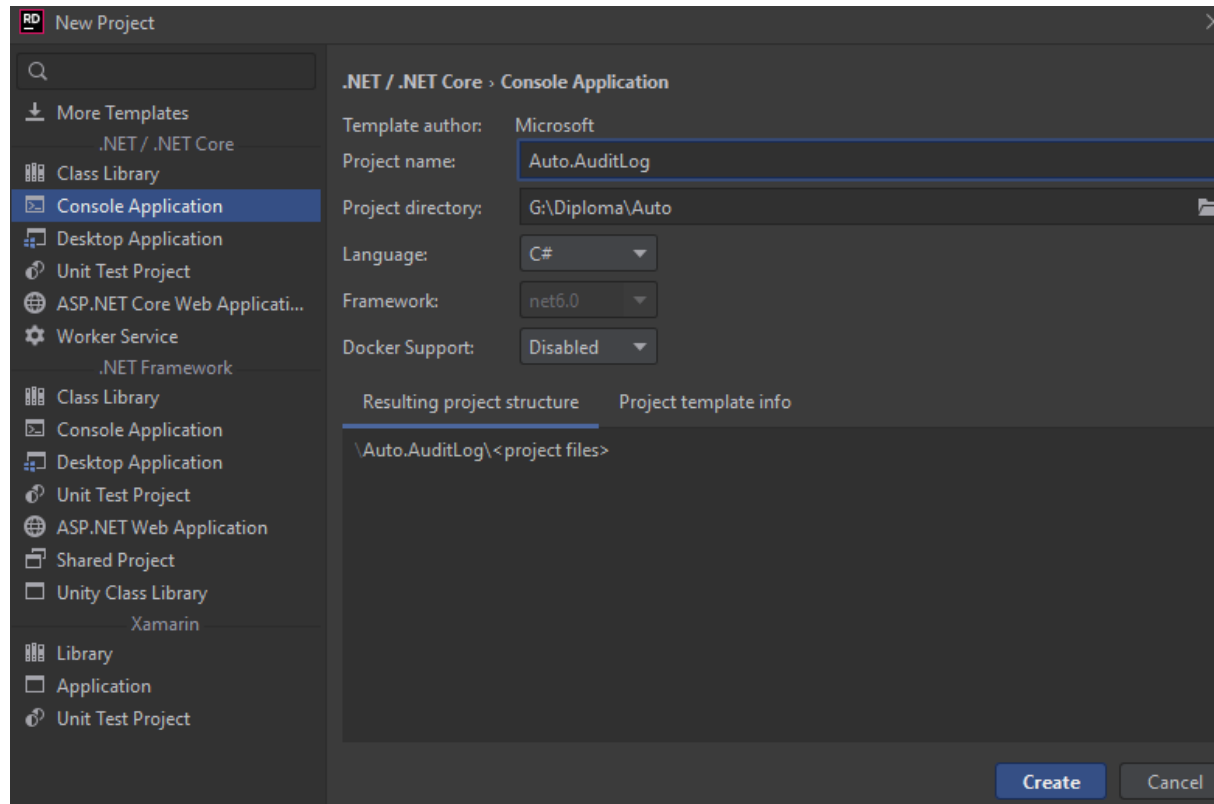
```
public class NewVehicleMessage {  
    public string Registration { get; set; }  
    public string ModelName { get; set; }  
    public string ManufacturerName { get;  
set; }  
    public string Color { get; set; }  
    public int Year { get; set; }  
    public DateTimeOffset CreatedAt { get;  
set; }  
}
```

```
public class NewVehiclePriceMessage : NewVehicleMessage {  
    public int Price { get; set; }  
    public string CurrencyCode { get; set; }  
  
    public NewVehiclePriceMessage() {  
    }  
  
    public NewVehiclePriceMessage(NewVehicleMessage  
vehicle, int price, string currencyCode) {  
        this.Registration = vehicle.Registration;  
        this.Color = vehicle.Color;  
        this.Year = vehicle.Year;  
        this.ManufacturerName = vehicle.ManufacturerName;  
        this.ModelName = vehicle.ModelName;  
        this.Price = price;  
        this.CurrencyCode = currencyCode;  
    }  
}
```



Создадим простое консольное приложение

Создайте консольное приложение Auto.AuditLog:





Создадим простое консольное приложение

Через NuGet подключите библиотеки

- *EasyNetQ*
- *Microsoft.Extensions.Configuration*
- *Microsoft.Extensions.Configuration.FileExtensions*
- *Microsoft.Extensions.Configuration.Json*
- *Microsoft.Extensions.Configuration.EnvironmentVariables*



Подключение

Чтобы подключиться к этому экземпляру RabbitMQ с помощью EasyNetQ, используйте эту строку подключения (создадим appsettings.json):

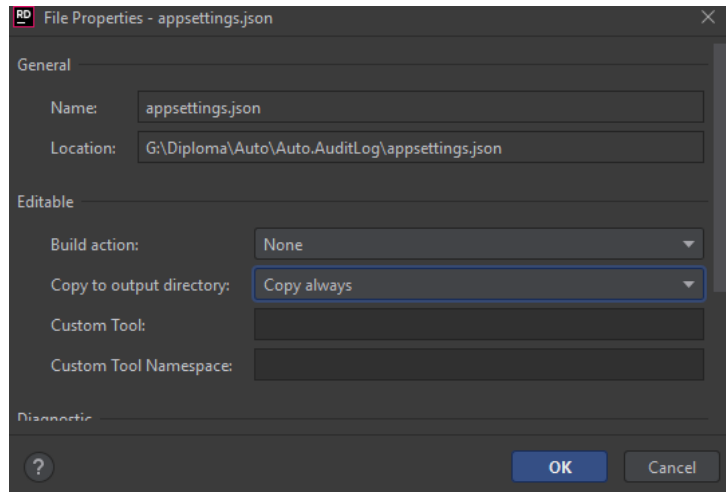
```
{
  "ConnectionStrings": {
    "AutoRabbitMQ": "amqp://user:rabbitmq@localhost:5672"
  }
}
```



Подключение

Чтобы подключиться к этому экземпляру RabbitMQ с помощью EasyNetQ, используйте эту строку подключения (создадим appsettings.json):

```
{
  "ConnectionStrings": {
    "AutoRabbitMQ": "amqp://user:rabbitmq@localhost:5672"
  }
}
```





Подключение к шине

```
static async Task Main(string[] args) {  
    Console.WriteLine("Starting Auto.AuditLog");  
    var amqp:string? = config.GetConnectionString(name: "AutoRabbitMQ");  
    using var bus = RabbitHutch.CreateBus(amqp);  
    Console.WriteLine("Connected to bus! Listening for newVehicleMessages");  
    var subscriberId = $"Auto.AuditLog@{Environment.MachineName}";  
    await bus.PubSub.SubscribeAsync<NewVehicleMessage>(subscriberId, HandleNewVehicleMessage);  
    Console.ReadLine();  
}
```



Обработка сообщений

```
private static void HandleNewVehicleMessage(NewVehicleMessage nvm) {  
    var csvRow:string =  
        $"{nvm.Registration},{nvm.ManufacturerName},{nvm.ModelName},{nvm.Year},{nvm.Color},{nvm.CreatedAt:0}";  
    Console.WriteLine(csvRow);  
}
```



Подключение WebApi к шине сообщений

- Добавим некоторый код для публикации NewVehicleMessage при добавлении нового транспортного средства на платформу.
- Нам нужно зарегистрировать IBus как зависимость, поэтому подключите пакет EasyNetQ к проекту и добавьте в Program.cs:

```
// Add RabbitMQ support
var bus =
RabbitHutch.CreateBus(builder.Configuration.GetConnectionString("AutoRabbitMQ"));
builder.Services.AddSingleton<IBus>(bus);
```

- Вам также необходимо добавить строку подключения RabbitMQ в appsettings.json:

```
"ConnectionStrings": {
  "AutoRabbitMQ": "amqp://user:rabbitmq@localhost:5672"
}
```



Подключение WebApi к шине сообщений

- Далее в контроллере добавим шину:

```
[Route(template: "api/[controller]")]
[ApiController]
public class VehiclesController : ControllerBase
{
    private readonly IAutoStorage _db;
    private readonly IBus _bus;

    public VehiclesController(IAutoStorage db, IBus bus)
    {
        this._db = db;
        this._bus = bus;
    }
}
```



Подключение WebApi к шине сообщений

- Создадим вспомогательный метод в HAL:

```
public static NewVehicleMessage ToMessage(this Vehicle vehicle) {  
    var message = new NewVehicleMessage() {  
        Registration = vehicle.Registration,  
        Color = vehicle.Color,  
        ManufacturerName = vehicle?.VehicleModel?.Manufacturer?.Name,  
        ModelName = vehicle?.VehicleModel?.Name,  
        Year = vehicle.Year,  
        CreatedAt = DateTimeOffset.UtcNow,  
    };  
    return message;  
}
```



Подключение WebApi к шине сообщений

- Создадим приватный метод для публикации сообщения:

```
private async Task PublishNewVehicleMessage(Vehicle vehicle) {  
    var message = vehicle.ToMessage();  
    await _bus.PubSub.PublishAsync(message);  
}
```



Подключение WebApi к шине сообщений

- И вызовем при создании объявления:

```
var vehicle = new Vehicle
{
    Registration = dto.Registration,
    Color = dto.Color,
    Year = dto.Year,
    VehicleModel = vehicleModel
};
_db.CreateVehicle(vehicle);
await PublishNewVehicleMessage(vehicle);
return Created(uri: $"/api/vehicles/{vehicle.Registration}", value: vehicle.ToResource());
```



Стартуем!

- Стартуем WebAPI
- Стартуем AuditLog

POST /api/vehicles/{id}

Parameters

Cancel

Reset

Name	Description
id * required string (path)	ABC1234

Request body

application/json

```
{  "registration": "ABC1234",  "color": "Green",  "year": 1976,  "modelCode": "volkswagen-beetle"}
```

```
{  "registration": "ABC1234",  "color": "Green",  "year": 1976,  "modelCode": "volkswagen-beetle"}
```

```
G:/Diploma/Auto/Auto.AuditLog/bin/Debug/net6.0/Auto.AuditLog.exe
Starting Auto.AuditLog
Connected to bus! Listening for newVehicleMessages
ABC1234,VOLKSWAGEN,BEETLE,1976,Green,2022-03-21T19:53:37.3013079+00:00
```



Будут вопросы? Не стесняйтесь!

